

Results:

Overall Accuracy:

Model	Solver	Avg Acc	All Correct
Llama-3.1-8B	CoT	0.00%	0.00%
Llama-3.1-8B	Tool	1.24%	0.00%
Qwen2.5-32B	CoT	6.20%	4.13%
Qwen2.5-32B	Tool	10.74%	4.13%

Tool Calling Delta (Tool - CoT):

Model	Avg Acc	All Correct
Llama-3.1-8B	+1.24%	+0.00%
Qwen2.5-32B	+4.54%	+0.00%

Error Breakdown:

Model	Solver	Correct	Wrong Sol	Bad Format	Bad Length	No Box	Says None
Llama-3.1-8B	CoT	0.0%	48.4%	9.9%	24.8%	14.9%	0.4%
Llama-3.1-8B	Tool	1.2%	51.2%	9.9%	20.2%	7.0%	7.4%
Qwen2.5-32B	CoT	6.2%	68.2%	7.4%	15.3%	0.4%	0.0%
Qwen2.5-32B	Tool	10.7%	57.0%	5.4%	15.3%	1.7%	3.7%

Per Category Accuracy:

Model	Solver	Number Theory	Combinatorics	Algebra	Geometry
Llama-3.1-8B	CoT	0.0%	0.0%	0.0%	0.0%
Llama-3.1-8B	Tool	2.8%	0.0%	0.0%	0.0%
Qwen2.5-32B	CoT	8.5%	1.2%	7.5%	0.0%
Qwen2.5-32B	Tool	13.2%	7.3%	10.0%	0.0%

Key takeaways:

- Tool calling helps both models but much more for Qwen-32B (+4.55% avg acc).
- Llama-8B basically can't solve these problems with or without tools.
- Geometry is 0% for all models/solvers so it is likely the hardest category.

Which tools were implemented:

Tool:	Purpose:
Python	General purpose python execution
Z3	SMT solver
OR-Tools	Combinatorial optimization
MiniZinc	Constraint programming
CryptoMiniSat	SAT solver
GA{	Group.algebra computation system
NetworkX	Graph theory
SymPy	Symbolic math
SageMath	Advance math computation
Lean	Formal proof verification

Routing by problem category:

- NUMBER_THEORY → Z3, CryptoMiniSat, SymPy, GAP, Python
- COMBINATORICS → OR-Tools, MiniZinc, Z3, NetworkX, Python
- ALGEBRA → SymPy, SageMath, Z3, Python
- GEOMETRY → SymPy, SageMath, Python
- PUZZLE → OR-Tools, MiniZinc, Z3, Python

How tool calling is done:

1. Problem tags drive tool selection so `get_tools_for_problem()` maps each problem's Tag (ex: COMBINATORICS) to a ranked list of tool instances.
2. Each tool is a `BaseTool` subclass so it defines an OpenAI style function schema and an `execute()` method that runs the actual solver (Z3, SymPy, OR-Tools, etc.) in a subprocess with a timeout.
3. `ToolAPIQuery` wraps the LLM call and it sends the problem + tool schemas to the model, then runs a multi-turn loop: if the model emits a tool call, it executes it and feeds the result back.
4. `ToolSolver` orchestrates as it calls `get_tools_for_problem()`, builds a `ToolAPIQuery`, and drives the conversation until the model produces a final answer.
5. The Python tool is always the fallback and included last in every category so the model can always write arbitrary code if specialized tools aren't sufficient.

In simple terms:

1. Each tool defines a name, description, and input schema (like a function signature) that gets sent to the model alongside the problem.
2. The model reads those descriptions and decides which tool to call and with what code/input and it replies with a structured tool-call instead of plain text.
3. The framework executes that call, returns the output to the model, and the model uses the result to produce its final answer.

Models used:

- Llama 8b
- Qwen 32b